

《现代C++编程实现》电子版: <https://leanpub.com/c01>

第9章 继承与派生

Derived class

YouTube频道: [hwdong](https://www.youtube.com/hwdong)

博客: hwdong-net.github.io

B站: [hw-dong](https://www.bilibili.com/video/av1000000000)

继承和派生

10.1 继承与派生概述

- **继承** (Inheritance) 是实现代码复用、功能扩展和类型层次化的核心机制。
- 继承通过定义**派生类** (Derived Class) 来实现, 新类从已有类 (称为**基类**, Base Class) 派生, 继承其成员 (数据和函数), 并可添加或修改特有功能。

继承与派生

- C++ 中的类用于描述现实世界的概念，这些概念常具有层次关系。例如，“**跑车**（SportsCar）”是一种**特殊的**“**汽车**（Car）”，它继承了汽车的通用属性（如速度、燃油）并添加独特特性（如高加速性能）。
- 两个独立类描述 Car 和 SportsCar：

```
class Car { /* ... */ };  
class SportsCar { /* ... */ };
```

编译器无法识别它们之间的这种 is-a 关系。

继承与派生

- 继承通过定义派生类实现，派生类从基类继承属性和方法，并可扩展或重定义功能。例如，SportsCar 可从 Car 派生：

```
class Car {  
    // 汽车的共有属性和行为  
};  
  
class SportsCar : public Car {  
    // SportsCar在继承Car的所有属性基础上，增加了自己的特有属性和方法  
};
```

派生类清楚表达了类之间的继承关系

```
class Car {  
    // 汽车的共有属性和行为  
};  
  
class SportsCar : public Car {  
    // SportsCar在继承Car的所有属性基础上，增加了自己的特有属性和方法  
};
```

- 在SportsCar类声明中，冒号 (:) 后面的public Car表示SportsCar通过公开继承自动获得Car的所有公共和保护（protected）成员。public关键字控制了Car的成员在SportsCar中的可见性（详见后续讨论）。

```
class Car {  
    // 汽车的共有属性和行为  
};  
  
class SportsCar : public Car {  
    // SportsCar在继承Car的所有属性基础上，增加了自己的特有属性和方法  
};
```

- 由于SportsCar继承了Car，无需在SportsCar中重复编写Car中已有的代码，只需聚焦跑车的特有功能。这种继承关系促进了**代码复用，提升了编程效率，增强了程序可靠性，并降低了错误和不一致的风险。**

```
class Car {  
    // 汽车的共有属性和行为  
};  
  
class SportsCar : public Car {  
    // SportsCar在继承Car的所有属性基础上，增加了自己的特有属性和方法  
};
```

- 通过定义派生类SportsCar，C++清楚地表达了SportsCar是Car的继承（is-a）关系，从而在程序中体现类之间概念的层次结构。

10.2 继承与组合：is-a 与 has-a

- is a 表示概念之间的**继承**关系
SportsCar is a Car
- has a 表示对象之间的**组合**关系
Car has a Engine

is-a

- 继承用于表达类之间的 **is-a**（是一种） 关系，即派生类（子类）是基类（父类）的特化形式。
- **is-a** 关系意味着派生类继承基类的属性和行为，并可根据自身需求扩展或重定义功能。
- 跑车（**SportsCar**）是一种汽车（**Car**）：SportsCar 继承 Car 的速度、行驶方法，并添加涡轮增压功能。
- 经理（**Manager**）是一种雇员（**Employee**）：Manager 继承 Employee 的考勤、薪资功能，并扩展团队管理职责。

```
class Car {  
protected:  
    double speed{0.0};  
public:  
    void drive() { /* 行驶逻辑 */ }  
};  
  
class SportsCar : public Car {  
private:  
    double acceleration{5.0};  
public:  
    void turboBoost() { /* 加速逻辑 */ }  
};
```

```
class Employee {  
protected:  
    string name;  
public:  
    void checkIn() { /* 打卡逻辑 */ }  
};  
  
class Manager : public Employee {  
    vector<Employee> employees;  
public:  
    void manageTeam() { /* 管理团队逻辑 */ }  
};
```

Has a

- **has-a**（具有） 关系表示一个类对象包含另一个类的对象，描述整体与部分的结构。
 - 汽车具有引擎（Car has-a Engine）。
 - 日期包含年、月、日（Date has-a year, month, day）。
- Has a关系通过 组合（**composition**） 实现，即将一个类的对象作为另一个类的成员变量

Has a

- Has a关系通过 组合（**composition**） 实现，即将一个类的对象作为另一个类的成员变量。

```
class Engine {  
    public:  
        void start() { /* 启动引擎 */ }  
};  
class Car {  
    private:  
        Engine engine; // 组合: Car 包含 Engine  
        double speed;  
    public:  
        void drive() {  
            engine.start(); // 使用 Engine 的功能  
            /* 行驶逻辑 */  
        }  
};
```

```
class Engine {
public:
    void start() { /* 启动引擎 */ }
};

class Car {
private:
    Engine engine; // 组合: Car 包含 Engine
    double speed;
public:
    void drive() {
        engine.start(); // 使用 Engine 的功能
        /* 行驶逻辑 */
    }
};
```

- **组合实现**: Car 类将 Engine 对象作为成员变量, 体现“汽车具有引擎”。
- **功能调用**: Car 通过 engine 访问 Engine 的方法 (如 start), 无需继承。

10.3 派生类的定义

派生类的基本定义格式为：

```
class 派生类名 : public 基类名 {  
    // 派生类的成员（数据成员和成员函数）  
};
```

• Car类

```
class Car {
protected:
    double speed{0.0};    // 汽车当前速度（单位：km/h）
    double fuel{100.0};   // 汽车燃油量（单位：L）
public:
    Car(double s = 0.0, double f = 100.0) : speed(s), fuel(f) {}

    // 驾驶汽车一定距离（简单模拟）
    void drive(double distance) {
        cout << "行驶了 " << distance << " 公里.\n";
    }

    // 显示汽车状态
    void showStatus() {
        cout << "当前速度: " << speed << " km/h, 燃油: " << fuel << " L\n";
    }
};
```

• SportsCar类

```
class SportsCar : public Car {
    double acceleration{5.0}; // 跑车特有的加速性能（单位：m/s2）
public:
    SportsCar(double s = 0.0, double f = 100.0, double a = 5.0)
        : Car(s, f), acceleration(a) {}

    // 跑车独有的涡轮增压功能，提高加速效果
    void turboBoost() {
        cout << "启动涡轮增压，加速性能大幅提升！\n";
        speed += acceleration * 2; // 模拟加速效果
    }

    // 重新定义showStatus()，同时显示跑车特有的信息
    void showStatus() {
        // 可选择调用基类的showStatus()函数
        Car::showStatus();
        cout << "加速度： " << acceleration << " m/s2\n";
    }
};
```

- 测试，SportsCar对象是否继承了Car的属性和方法：

```
int main() {  
    SportsCar sc;  
    sc.drive(20);           // 继承的 drive  
    sc.showStatus();        // 重定义的 showStatus  
    sc.turboBoost();        // 特有方法  
    sc.showStatus();        // 更新后的状态  
    return 0;  
}
```

行驶 20 公里

速度：0 km/h，燃油：100 L

加速度：5 m/s²

启动涡轮增压，速度提升

速度：10 km/h，燃油：100 L

加速度：5 m/s²

```
class SportsCar : public Car {  
    double acceleration{5.0}; // 跑车特有的加速性能（单位：m/s2）  
public:  
    SportsCar(double s = 0.0, double f = 100.0, double a = 5.0)  
        : Car(s, f), acceleration(a) {}  
  
    // 跑车独有的涡轮增压功能，提高加速效果  
    void turboBoost() {  
        cout << "启动涡轮增压，加速性能大幅提升！\n";  
        speed += acceleration * 2; // 模拟加速效果  
    }  
  
    // 重新定义showStatus()，同时显示跑车特有的信息  
    void showStatus() {  
        // 可选择调用基类的showStatus()函数  
        Car::showStatus();  
        cout << "加速度：" << acceleration << " m/s2\n";  
    }  
};
```

10.4 成员的隐藏 (Hide)

- 在继承关系中，派生类可以定义与基类同名的成员（变量或函数）。当发生这种情况时，派生类的同名成员会隐藏（hide）基类的对应成员。

如SportsCar中定义了和基类Car同名的成员函数showStatus()。

10.4 成员的隐藏 (Hide)

- 隐藏意味着：通过派生类对象访问该成员时，默认使用的是派生类中定义的成员，基类的同名成员被屏蔽，无法直接访问。如 `sc.showStatus()` 调用的是 `SportsCar` 的 `showStatus()`。

```
class Car {  
public:  
    void showStatus() { std::cout << "Car status\n"; }  
};  
  
class SportsCar : public Car {  
public:  
    void showStatus() { std::cout << "SportsCar status\n"; } // 隐藏基类的showStatus  
};  
  
int main() {  
    SportsCar sc;  
    sc.showStatus(); // 输出 "SportsCar status"  
}
```

10.4 成员的隐藏 (Hide)

- **注意：**隐藏是指派生类成员遮盖了基类成员，这与 **函数重载** 不同，后者是在**同一个作用域**中定义多个同名但参数不同的函数。

11.4.1 同名变量隐藏

- 若派生类定义了与基类同名的变量，派生类的变量会隐藏基类的变量。两者完全独立，占用不同内存，互不影响，且类型可不同。

```
class Car {  
public:  
    int speed = 100; // 基类中的 speed  
};  
  
class SportsCar : public Car {  
public:  
    double speed = 200.5; // 派生类中的 speed (类型不同)  
    void showSpeed() {  
        cout << "SportsCar speed: " << speed << endl;  
        cout << "Car speed: " << Car::speed << endl;  
    }  
};
```

```
class Car {
public:
    int speed = 100;    // 基类中的 speed
};

class SportsCar : public Car {
public:
    double speed = 200.5;    // 派生类中的 speed (类型不同)
    void showSpeed() {
        cout << "SportsCar speed: " << speed << endl;
        cout << "Car speed: " << Car::speed << endl;
    }
};

int main() {
    SportsCar sc;
    sc.showSpeed();
    sc.speed = 150;    // sc.speed是派生类的speed
    return 0;
}
```

```
class Car {
public:
    int speed = 100;    // 基类中的 speed
};
class SportsCar : public Car {
public:
    double speed = 200.5;    // 派生类中的 speed (类型不同)
    void showSpeed() {
        cout << "SportsCar speed: " << speed << endl;
        cout << "Car speed: " << Car::speed << endl;
    }
};
```

- 基类 `Car` 中的 `speed` 是 `int` 类型，而派生类 `SportsCar` 中的 `speed` 是 `double` 类型。两者，不会相互影响。
- 派生类的 `speed` 完全隐藏了基类的 `speed`：
 - 在派生类中访问 `speed` 时，默认访问的是派生类的成员。
 - 通过 `SportsCar` 对象访问 `speed` 默认获取派生类的值。
- 如果要访问基类的 `speed`，需要使用 `Car::speed` 进行显式指定。

1.4.2 同名函数隐藏

- 若派生类定义了与基类同名的函数，派生类的函数会隐藏基类的同名函数，无论参数列表是否相同。基类函数不可直接访问，除非通过作用域限定符或 using 声明引入。

```
class Car {  
public:  
    void showStatus() {  
        cout << "Car status: Normal" << endl;  
    }  
};  
  
class SportsCar : public Car {  
public:  
    void showStatus() {  
        cout << "SportsCar status: High Speed" << endl;  
    }  
};
```

```
class Car {
public:
    void showStatus() {
        cout << "Car status: Normal" << endl;
    }
};

class SportsCar : public Car {
public:
    void showStatus() {
        cout << "SportsCar status: High Speed" << endl;
    }
};

int main() {
    SportsCar sc;
    sc.showStatus(); // 调用的是 SportsCar 的 showStatus()
    sc.Car::showStatus(); // 显式调用 Car 的 showStatus()
    return 0;
}
```

1.4.2 同名函数隐藏

- 同名函数隐藏的规则与重载不同：函数重载要求函数名相同，但参数列表必须不同，而成员隐藏则只看函数名，不考虑参数列表。
- 即使基类和派生类中的同名函数参数列表不同，基类的版本也会被隐藏。

10.5 继承方式与访问控制

- 在 C++ 中，类的成员可以使用 `private`、`protected` 和 `public` 关键字来控制其可见性和访问权限。同样地，继承时也可以通过这三个关键字指定继承方式，它们决定了基类成员在派生类中的可见性及对外可访问性。

10.5.1. 默认继承方式

继承方式的默认行为取决于派生类的定义方式：

- class 定义的派生类默认采用 private 继承。

```
class Base1 {  
    // 基类内容  
};  
class Derived1 : Base1 { // 等价于 class Derived1 : private Base1  
    // 派生类内容  
};
```

10.5.1. 默认继承方式

继承方式的默认行为取决于派生类的定义方式：

- class 定义的派生类默认采用 private 继承。
- struct 定义的派生类默认采用 public 继承。

```
struct Base2 {  
    // 基类内容  
};  
struct Derived2 : Base2 { // 等价于 struct Derived2 : public Base2  
    // 派生类内容  
};
```

10.5.2 继承方式类型

- **public** 继承：基类的 public 成员保持 public，protected 成员保持 protected。常用于 *is-a* 关系，例如：SportsCar 是 Car，派生类继承并扩展基类的公开接口。

10.5.2 继承方式类型

- **protected 继承**：基类的public和protected成员在派生类中变为protected。

这意味着派生类的对象无法直接从外部调用这些成员，但派生类的成员函数和其派生类仍可访问这些成员。例如，RestrictedCar通过protected继承Car，限制外部对Car接口的访问。适用于在继承层次中保护基类功能，仅允许派生类及其继承者使用。

```
class Car {
public:
    void drive() { std::cout << "Driving\n"; }
protected:
    void engineStatus() { std::cout << "Engine OK\n"; }
};

class RestrictedCar : protected Car {
public:
    void check() {
        drive();           // OK, 成员函数可访问
        engineStatus();    // OK, 成员函数可访问
    }
};

int main() {
    RestrictedCar rc;
    // rc.drive(); // 错误, drive在RestrictedCar中是protected
    rc.check();       // OK, 调用成员函数
}
```

10.5.2 继承方式类型

- **private** 继承：基类的 public 和 protected 成员在派生类中变为 private，只能在派生类的成员函数中访问，外部和进一步的派生类无法访问。

private 继承用于基于基类的内部实现重用，派生类利用基类的功能实现自身逻辑，但不暴露基类的接口，也不体现 *is-a* 关系。

```
class Car {
public:
    void drive() { std::cout << "Driving\n"; }
protected:
    void engineStatus() { std::cout << "Engine OK\n"; }
};

class PrivateCar : private Car {
public:
    void operate() {
        drive();           // OK, 成员函数可访问
        engineStatus();    // OK, 成员函数可访问
        std::cout << "Operating privately\n";
    }
};

int main() {
    PrivateCar pc;
    // pc.drive(); // 错误, drive在PrivateCar中是private
    pc.operate();    // OK, 调用派生类自己的接口
}
```

10.5.3 访问控制规则

- 基类成员的可见性（private、protected、public）与派生类的继承方式（private、protected、public）结合后，决定了基类成员在派生类中的可见性和对外的可访问性。

基类成员	private 继承	protected 继承	public 继承
private	不可见	不可见	不可见
protected	private	protected	protected
public	private	protected	public

基类成员	private 继承	protected 继承	public 继承
private	不可见	不可见	不可见
protected	private	protected	protected
public	private	protected	public

- **public 继承**：保持基类成员的对外可见性。
- **protected 继承**：基类的 public 和 protected 成员变为 protected，对外不可见。
- **private 继承**：基类的 public 和 protected 成员变为 private，对外不可见。
- **基类的private 成员**：无论何种继承方式，基类的 private 成员在派生类中始终不可见。

10.5.4 可见性与存在性

- 不可见 $\neq$ 不存在：基类的 `private` 成员虽然在派生类中不可见，但在派生类对象中依然存在。例如，基类的 `private` 数据成员会在派生类对象中分配内存，只不过派生类无法直接访问它。
- 可以通过 基类的公有或受保护的成员函数 间接访问基类的 `private` 成员。

```
class Base {  
private:  
    int privateVar;  
protected:  
    int protectedVar;  
public:  
    int publicVar;  
    int getPrivateVar() const { return privateVar; }  
};
```

```
class Derived : public Base {  
public:  
    void show() {  
        // cout << privateVar; // 错误: 不可访问  
        cout << protectedVar; // 可以访问  
        cout << publicVar;     // 可以访问  
        cout << getPrivateVar(); // 间接访问  
    }  
};
```

10.6 对象切割

- 当派生类对象被赋值给基类对象，或以值传递方式作为基类类型参数时，会发生**对象切割**（Object Slicing）。此时，派生类对象的基类部分被复制到目标对象，派生类特有的成员（变量和函数）被丢弃，导致只能访问基类内容。切割后的对象成为纯粹的基类对象，丧失派生类特性。

```
class Car {
protected:
    std::string model;
public:
    Car(const std::string& model) : model(model) {}
    void print() { std::cout << "Car: " << model << '\n'; }
};

class SportsCar : public Car {
    double maxSpeed;
public:
    SportsCar(const std::string& model, double speed) : Car(model), maxSpeed(speed) {}
    void print() {
        std::cout << "SportsCar: ";
        Car::print();
        std::cout << "Max Speed: " << maxSpeed << '\n';
    }
};
```

```
int main() {  
    Car car("Sedan");  
    SportsCar sportsCar("Porsche", 300.0);  
  
    car.print();           // 输出: Car: Sedan  
    sportsCar.print();     // 输出: SportsCar: Car: Porsche   Max Speed: 300  
  
    car = sportsCar;       // 对象切割  
    car.print();           // 输出: Car: Porsche  
}
```

- 在 `car = sportsCar;` 中, `SportsCar` 对象 `sportsCar` 被赋值给 `Car` 对象 `car`, 发生**对象切割**。
- 只有 `sportsCar` 的基类部分 (即 `Car` 的 `model` 成员) 被复制到 `car`, 派生类的特有成员 (如 `maxSpeed` 和 `SportsCar::print()`) 被丢弃。
- 赋值后, `car` 是一个纯粹的 `Car` 对象, 调用 `car.print()` 只输出基类信息 (如 `Car: Porsche`), 无法访问 `SportsCar` 的特性。

10.6.3. 内存布局与对象切割

- 派生类 SportsCar 的内存布局:

std::string model	<- 基类 Car 的成员
double maxSpeed	<- 派生类 SportsCar 的扩展部分

- 基类 Car 的内存布局:

std::string model

10.6.3. 内存布局与对象切割

- 切割后 car 的内存布局:

```
| std::string model | <- 仅保留基类部分
```

10.6.4. 对象切割的场景

对象切割常见于：

- 1. 赋值：派生类对象赋值给基类对象（如 `car = sportsCar;`）。
- 2. 值传递：派生类对象以值传递方式传给基类形参。

```
void print(Car car) {  
    car.print(); // 只调用 Car::print()  
}  
  
int main() {  
    SportsCar sportsCar("Porsche", 320.0);  
    print(sportsCar); // 对象切割发生  
    return 0;  
}
```

10.6.5. 避免对象切割

要避免对象切割，应使用基类引用或指针传递派生类对象：

```
void print(Car& car) {  
    car.print(); // 调用的是 Car::print(), 不会自动调用 SportsCar::print()  
  
    std::cout << "--- After cast ---\n";  
    static_cast<SportsCar&>(car).print(); // 强制转换为 SportsCar 类型后,  
                                           // 调用 SportsCar::print()  
}  
  
int main() {  
    SportsCar sportsCar("Porsche", 320.0);  
    print(sportsCar); // 不发生对象切割  
}
```

10.7 向上类型转换与向下类型转换

- 派生类对象本质上包含基类部分，因此基类指针或基类引用可指向派生类对象（**向上类型转换**）。反之，将基类指针或引用强制转换为派生类指针或引用（**向下类型转换**）则需谨慎，以确保类型安全。

10.7.1 向上类型转换 (Upcasting)

定义： 将派生类指针或引用转换为基类指针或引用。

特点：

- 隐式且安全： 派生类对象包含基类部分，转换无需显式操作。
- 访问限制： 基类指针或引用只能访问基类成员，无法访问派生类扩展的成员。

```
class B {  
protected:  
    int b;  
public:  
    B(int b = 0) : b(b) {}  
    void print() const { cout << "B: " << b << "\n"; }  
};
```

```
class D : public B {  
private:  
    double d;  
public:  
    D(double d = 0) : B(0), d(d) {}  
    void print() const {  
        cout << "D: ";  
        B::print();  
        cout << "\t" << d << "\n";  
    }  
};
```

```
int main() {  
    D d(3.14);  
    B* p = &d;    // 向上转换: 基类指针  
    B& r = d;     // 向上转换: 基类引用  
    p->print();    // 调用 B::print  
    r.print();     // 调用 B::print  
    return 0;  
}
```

输出:

B: 0

B: 0

10.7.2. 向下类型转换 (Downcasting)

定义： 将基类指针或引用显式转换为派生类指针或引用。

特点：

- 显式转换：需使用 `static_cast` 或 `dynamic_cast`。
- 风险：若基类指针或引用不指向派生类对象，转换可能导致未定义行为。

- 安全的向下类型转换

```
int main() {  
    D d(3.14);  
    B* p = &d;           // 向上转换  
    D* q = static_cast<D*>(p); // 向下转换  
    q->print();           // 调用 D::print  
    return 0;  
}
```

- 不安全的向下类型转换

```
int main() {  
    B b(1);  
    B* p = &b;           // 指向基类对象  
    D* q = static_cast<D*>(p); // 不安全向下转换  
    q->print();           // 未定义行为  
    return 0;  
}
```

- 使用 `dynamic_cast` 可提高向下转换安全性（需基类含虚函数，详见多态章节）：

```
class B {
public:
    virtual void print() const { // virtual声明该函数是虚函数
        cout << "B: " << b << "\n";
    }
    // 其他成员同上
};

int main() {
    B* p = new D(3.14);
    D* q = dynamic_cast<D*>(p); // 安全向下转换
    if (q) q->print();          // 检查转换成功
    else cout << "Invalid cast\n";
    delete p;
    return 0;
}
```

- `dynamic_cast` 检查 `p` 是否指向 `D` 对象，失败时返回 `nullptr`，避免未定义行为。

```
class B {
public:
    virtual void print() const { // virtual声明该函数是虚函数
        cout << "B: " << b << "\n";
    }
    // 其他成员同上
};

int main() {
    B* p = new D(3.14);
    D* q = dynamic_cast<D*>(p); // 安全向下转换
    if (q) q->print();          // 检查转换成功
    else cout << "Invalid cast\n";
    delete p;
    return 0;
}
```

10.8 派生类的构造函数、析构函数与拷贝构造函数

10.8.1 构造函数与析构函数

- 在构造派生类对象时，基类部分优先于派生类部分初始化。

```
class B {  
private:  
    int b = 0; // 使用初始化列表  
public:  
    B() { cout << "B类的默认构造函数" << endl; }  
};  
  
class D : public B {  
private:  
    double d = 2.5; // 使用初始化列表  
public:  
    D() { cout << "D类的默认构造函数" << endl; }  
};
```

```
int main() {  
    D d;  
}
```

B类的默认构造函数
D类的默认构造函数

10.8.1 构造函数与析构函数

- 在对象销毁时，析构过程与构造过程相反，先销毁派生类部分，再销毁基类部分。

```
class B {  
private:  
    int b = 0; // 使用初始化列表  
public:  
    B() { cout << "B类的默认构造函数" << endl; }  
    ~B() { cout << "B类的析构函数" << endl; }  
};  
  
class D : public B {  
private:  
    double d = 2.5; // 使用初始化列表  
public:  
    D() { cout << "D类的默认构造函数" << endl; }  
    ~D() { cout << "D类的析构函数" << endl; }  
};
```

```
int main() {  
    D d;  
}
```

B类的默认构造函数
D类的默认构造函数
D类的析构函数
B类的析构函数

构造与析构顺序

- **构造函数顺序**：从最顶层基类到最底层派生类。构造时，基类的成员先于派生类成员被初始化。顺序如下：
 1. **最顶层基类** → **逐层派生类** → **最底层派生类**。
- **析构函数顺序**：从最底层派生类到最顶层基类。析构时，派生类的成员先于基类成员被销毁。顺序如下：
 1. **最底层派生类析构函数** → **逐层基类析构函数** → **最顶层基类析构函数**。

例如，在多层继承 `A -> B -> D` 中，创建 `D` 对象时的构造顺序为：

`A` 类构造函数 → `B` 类构造函数 → `D` 类构造函数

销毁顺序为：

`D` 类析构函数 → `B` 类析构函数 → `A` 类析构函数

10.8. 2. 在初始化成员列表中调用基类构造函数

- 默认情况下，派生类会隐式调用基类的默认构造函数。如果基类没有默认构造函数，派生类必须在构造函数初始化列表中显式调用基类的其他构造函数，以确保基类部分正确初始化

```
class B {
private:
    int b = 0; // 使用初始化列表
public:
    B() { cout << "B类的默认构造函数" << endl; }
    B(int b) : b(b) { cout << "B类的构造函数" << endl; }
    ~B() { cout << "B类的析构函数" << endl; }
};

class D : public B {
private:
    double d = 2.5; // 使用初始化列表
public:
    D() : B(2) { cout << "D类的默认构造函数" << endl; } // 在构造函数初始化列表中调用基类构造函数
    ~D() { cout << "D类的析构函数" << endl; }
};
```

```
class B {
private:
    int b = 0; // 使用初始化列表
public:
    B() { cout << "B类的默认构造函数" << endl; }
    B(int b) : b(b) { cout << "B类的构造函数" << endl; }
    ~B() { cout << "B类的析构函数" << endl; }
};

class D : public B {
private:
    double d = 2.5; // 使用初始化列表
public:
    D() : B(2) { cout << "D类的默认构造函数" << endl; } // 在构造函数初始化列表中调用基类构造函数
    ~D() { cout << "D类的析构函数" << endl; }
};
```

```
int main() {
    D d;
}
```

B类的构造函数
D类的默认构造函数
D类的析构函数
B类的析构函数

- 错误用法：在构造函数体内调用基类构造函数

```
D() { B(2); cout << "D类的默认构造函数" << std::endl; }
```

在这种情况下，B(2) 创建了一个临时的 B 对象，但并未对 D 类对象的基类部分进行初始化。D 类的默认构造函数此时仍然调用 B 的默认构造函数对其基类部分进行构造。

- 若基类无默认构造函数，派生类需在初始化列表中显式调用基类构造函数并提供参数。

```
class B {
private:
    int b;
    string name;
public:
    B(int b, const string& n) : b(b), name(n) { cout << "B类的构造函数\n"; }
    ~B() { cout << "B类的析构函数\n"; }
};

class D : public B {
private:
    double d = 2.5;
public:
    D(double d, int b, const string& n) : B(b, n), d(d) { cout << "D类的构造函数\n"; }
    ~D() { cout << "D类的析构函数\n"; }
};
```

- 硬编码初始化方式：可在初始化列表中硬编码基类构造函数参数

```
D() : B(2, "name"), d(2.5) { cout << "D类的默认构造函数" << std::endl; }
```

但此方式使所有 D 对象使用相同基类初始化值，缺乏灵活性。建议通过派生类构造函数参数动态传递基类构造函数参数。

10.8.3 拷贝构造函数

- 当使用同一类的对象初始化一个新对象时，会自动调用拷贝构造函数。如果未显式定义拷贝构造函数，编译器将为类生成一个默认的拷贝构造函数。
- 若派生类的拷贝构造函数未显式调用基类的拷贝构造函数，编译器会调用**基类的默认构造函数**，而非拷贝构造函数。这可能导致基类子对象未正确初始化，尤其是当基类管理动态资源时，可能引发问题。

```
class B {
private:
    int b = 0; // 使用初始化列表
public:
    B() { cout << "B类的默认构造函数" << endl; }
    B(const B& b) { cout << "B类的拷贝构造函数" << endl; }
};

class D : public B {
private:
    double d = 2.5; // 使用初始化列表
public:
    D() { cout << "D类的默认构造函数" << endl; }
    D(const D& d) { cout << "D类的拷贝构造函数" << endl; }
};
```

```
int main() {
    D d, d2(d);
}
```

B类的默认构造函数
D类的默认构造函数
B类的默认构造函数
D类的拷贝构造函数

- 为确保基类子对象正确初始化，应在派生类拷贝构造函数的初始化列表中显式调用基类的拷贝构造函数：

```
class B {
private:
    int b = 0; // 使用初始化列表
public:
    B() { cout << "B类的默认构造函数" << endl; }
    B(const B& b) { cout << "B类的拷贝构造函数" << endl; }
};

class D : public B {
private:
    double d = 2.5; // 使用初始化列表
public:
    D() { cout << "D类的默认构造函数" << endl; }
    D(const D& d) : B(d) { // 在构造函数初始化列表中调用基类的拷贝构造函数
        cout << "D类的拷贝构造函数" << endl;
    }
};
```

10.9 虚函数与多态

- 在面向对象编程中，多态性（Polymorphism）是一个至关重要的概念，它允许不同的类通过相同的接口进行交互。
- C++ 中通过**虚函数**（virtual function）来实现运行时多态。虚函数机制使程序能够在运行时决定实际调用哪个函数，从而实现 **动态绑定**（dynamic binding），是构建灵活、可扩展系统的基础。

10.9.1 虚函数概述

- 虚函数是基类中用 **virtual** 关键字声明的成员函数，允许派生类通过重写（override）提供自己的实现。当通过基类指针或引用调用虚函数时，C++根据对象的实际类型（而非指针或引用的类型）在运行时选择调用基类或派生类的版本，实现动态绑定和多态。
- 在基类中用 virtual 关键字声明虚函数，派生类重写时建议使用 override 关键字以明确标识。

```
class Animal {
public:
    virtual void speak() { // 虚函数
        cout << "Animal speaks" << endl;
    }
};

class Dog : public Animal {
public:
    void speak() override { // 重写虚函数
        cout << "Dog barks" << endl;
    }
};

class Cat : public Animal {
public:
    void speak() override { // 重写虚函数
        cout << "Cat meows" << endl;
    }
};
```

```
int main() {
    Animal* animalPtr;

    Dog dog;
    Cat cat;

    // 基类指针指向派生类对象，发生多态
    animalPtr = &dog;
    animalPtr->speak(); // 输出: Dog barks

    animalPtr = &cat;
    animalPtr->speak(); // 输出: Cat meows

    return 0;
}
```

```

class Animal {
public:
    virtual void speak() { // 虚函数
        cout << "Animal speaks" << endl;
    }
};

class Dog : public Animal {
public:
    void speak() override { // 重写虚函数
        cout << "Dog barks" << endl;
    }
};

class Cat : public Animal {
public:
    void speak() override { // 重写虚函数
        cout << "Cat meows" << endl;
    }
};

```

```

int main() {
    Animal* animalPtr;

    Dog dog;
    Cat cat;

    // 基类指针指向派生类对象，发生多态
    animalPtr = &dog;
    animalPtr->speak(); // 输出: Dog barks

    animalPtr = &cat;
    animalPtr->speak(); // 输出: Cat meows

    return 0;
}

```

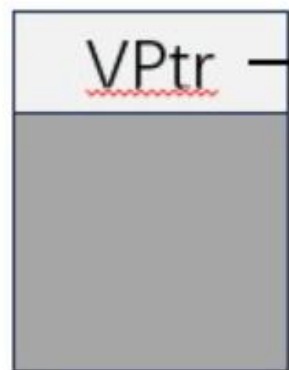
当我们使用基类指针 `animalPtr` 时，C++ 会根据它实际指向的对象类型（Dog 或 Cat）来调用对应类的 `speak()` 函数，这种决策是在运行时进行的，因此称为**动态绑定**（Dynamic Binding）。

10.9.2 虚函数的工作原理

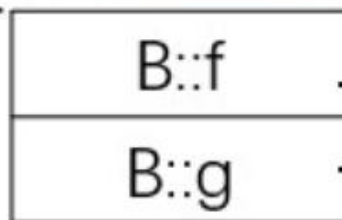
虚函数的实现依赖于 **虚函数表** (VTable) 和 **虚函数表指针** (VPtr) 机制：

- **虚函数表** (VTable)：每个包含虚函数的类都维护一张表，存储该类的虚函数地址。
- **虚函数表指针** (VPtr)：每个对象包含一个指向其所属类 VTable 的指针。当使用基类指针或引用调用虚函数时，程序通过 VPtr 查找 VTable，再调用正确的函数。

B的对象

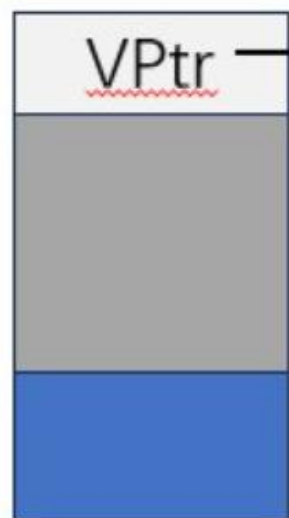


B的VTable

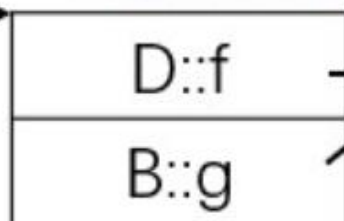


```
class B{  
    virtual void f(){...}  
    virtual void g(){...}  
};
```

D的对象



D的VTable



```
class D:public B{  
    virtual void f(){...}  
};
```

```
B *p = new D;  
P->f();
```

10.9.4 虚析构函数

当基类指针指向派生类对象时，如果**基类的析构函数不是虚函数**，则只会调用基类的析构函数，导致派生类的资源无法正确释放，可能引发**内存泄漏**。

```
class Base {
public:
    Base() { std::cout << "Base constructor" << std::endl; }
    ~Base() { std::cout << "Base destructor" << std::endl; } // 关键：虚析构函数
};

class Derived : public Base {
public:
    Derived() {
        data = new int[10]; // 分配一些内存
        std::cout << "Derived constructor" << std::endl;
    }
    ~Derived() {
        delete[] data; // 释放内存
        std::cout << "Derived destructor" << std::endl;
    }
private:
    int* data;
};
```

```
class Base {
public:
    Base() { std::cout << "Base constructor" << std::endl; }
    ~Base() { std::cout << "Base destructor" << std::endl; } // 关键：虚析构函数
};

class Derived : public Base {
public:
    Derived() {
        data = new int[10]; // 分配一些内存
        std::cout << "Derived constructor" << std::endl;
    }
    ~Derived() {
        delete[] data; // 释放内存
        std::cout << "Derived destructor" << std::endl;
    }
private:
    int* data;
};
```

```
int main() {
    Base* ptr = new Derived(); // 向上转型
    delete ptr; // 通过基类指针删除派生类对象

    return 0;
}
```

Base constructor
Derived constructor
Base destructor

解决方法是将基类的析构函数声明为虚函数，即将 Base 类的析构函数声明为虚函数，即

virtual ~Base() { ...}。

修改后的输出结果：

```
Base constructor  
Derived constructor  
Derived destructor  
Base destructor
```

如果一个类的基类析构函数是虚函数，那么这个类的析构函数即使不显式声明为 `virtual`，仍然是虚函数，但是，为了代码的可读性和清晰性，强烈建议在派生类的虚函数声明中使用 **override** 关键字，这样可以明确地表明该函数重写了基类的虚函数。

```
~Derived() override {  
    // ...  
}
```

10.9.4 编译时多态 vs 运行时多态

- **多态** (Polymorphism) 允许同一操作在不同对象上表现出不同行为。在 C++ 中, 多态性 (Polymorphism) 是面向对象编程的一个重要特性。C++ 中的多态分为**编译时多态** (静态多态) 和**运行时多态** (动态多态), 虽然它们都允许相同的函数名在不同的上下文中表现出不同的行为, 但它们的实现方式和使用场景有所不同。

1. 编译时多态（静态多态）

- 编译时多态在编译时确定函数调用版本，主要通过函数重载实现。函数重载允许在同一作用域内定义多个同名函数，但参数列表（类型、数量或顺序）不同，编译器根据函数签名选择调用版本。

2. 运行时多态（动态多态）

- 运行时多态在程序运行时根据对象实际类型决定调用哪个函数，通过虚函数和继承实现。基类指针或引用指向派生类对象时，调用虚函数会根据对象类型动态选择版本。

```
class Base {  
public:  
    virtual void speak() { cout << "Base speak" << endl; }  
    void speak(int x) { cout << "Base speak with " << x << endl; } // 重载函数  
};
```

```
class Derived : public Base {  
public:  
    void speak() override { cout << "Derived speak" << endl; }  
};
```

```
int main() {  
    Base* basePtr = new Derived();  
    basePtr->speak(); // 调用虚函数, 输出: Derived speak  
    basePtr->speak(10); // 调用重载函数, 输出: Base speak with 10  
    delete basePtr;  
    return 0;  
}
```

10.9.6 类型转换与 `dynamic_cast`

类型转换是处理多态性时非常重要的一部分。除了使用虚函数和继承来实现多态性外，类型转换使得我们能够在不同的类型之间进行转换，特别是在涉及到基类指针或引用指向派生类对象的情况下。C++提供 `static_cast`和 `dynamic_cast` 两种机制，分别适用于不同场景。

1. static_cast 与 dynamic_cast 对比

- **static_cast**: 用于在编译时进行类型转换，适用于已知类型关系的转换。它通常用于以下情况：
 - 基本类型之间的转换。
 - 具有明确继承关系的类之间的转换（向上转换是安全的，向下转换需要谨慎）。
 - 已知类型信息的指针或引用之间的转换。
 - `static_cast` 在向下转换时是不安全的，因为它不会检查指针或引用指向的实际对象类型。
- **dynamic_cast**: 用于运行时进行类型转换，特别是在处理多态对象时。
 - 在运行时通过运行时类型信息（`RTTI`）进行类型转换，仅适用于包含虚函数的类层次。
 - 向下转换失败时返回 `nullptr`（指针）或抛出 `std::bad_cast`（引用），确保安全性。

```
class Animal {
public:
    virtual void makeSound() { std::cout << "Animal sound\n"; }
    virtual ~Animal() = default;
};

class Dog : public Animal {
public:
    void makeSound() override { std::cout << "Woof!\n"; }
};

class Cat : public Animal {
public:
    void makeSound() override { std::cout << "Meow!\n"; }
};
```

```
int main() {
    Animal* animal = new Dog();

    // static_cast: 不安全向下转换
    Dog* dogStatic = static_cast<Dog*>(animal); // 正确
    dogStatic->makeSound();
    Cat* catStatic = static_cast<Cat*>(animal); // 不安全, 可能未定义行为
    catStatic->makeSound(); // 输出 woof! 或崩溃

    // dynamic_cast: 安全向下转换
    Dog* dogDynamic = dynamic_cast<Dog*>(animal); // 成功
    if (dogDynamic) dogDynamic->makeSound();
    Cat* catDynamic = dynamic_cast<Cat*>(animal); // 失败, 返回 nullptr
    if (catDynamic) {
        catDynamic->makeSound();
    } else {
        std::cout << "Failed to cast to Cat\n";
    }

    delete animal;
}
```

3. 对比表格

特性	static_cast	dynamic_cast
转换时机	编译时	运行时
类型检查	编译时检查，依赖程序员保证正确性	运行时检查，依赖RTTI
失败处理	无运行时检查，可能未定义行为	返回 nullptr 或抛出 std::bad_cast
适用场景	明确类型关系的转换	多态场景中需要安全转换

10.9.7 虚函数的一些语法规则

类体外定义虚函数

- 虚函数在类体内声明时需使用 `virtual` 关键字，但在类体外定义时不可重复使用 `virtual`。

```
class Person {  
protected:  
    string name{ "noname" };  
public:  
    Person(string n) :name(n) {}  
    virtual void print(); // 类体内的函数声明前使用 virtual  
};  
  
// 类体外定义时不能再写 virtual  
void Person::print() {  
    cout << name << '\n';  
}
```

虚函数签名和返回类型

- 派生类重写的虚函数必须与基类虚函数的**签名**（函数名、参数列表、const 修饰）完全相同。**返回类型**需相同或为协变类型（基类返回基类指针/引用，派生类返回派生类指针/引用）。

```
class B {  
public:  
    virtual B& f() { return *this; }  
    virtual int g() { std::cout << "g\n"; return 0; }  
};  
  
class D : public B {  
public:  
    D& f() { return *this; }  
    double g() { std::cout << "g\n"; return 0.; }  
};
```

使用 override 关键字

- override 关键字用于显式标记派生类重写基类的虚函数。它帮助编译器验证是否正确重写了基类的虚函数。如果虚函数名拼写错误或签名不匹配，编译器会报告错误。

```
class X {  
    public:  
        virtual void print() { }  
};  
  
class Y : public X {  
    public:  
        virtual void Print() override { } // 编译时会报错：未重写基类方法  
};
```

使用 final 关键字

- 虚函数可以一直被派生类继承下去，但有时希望在派生类的某个层次上终止虚函数的向下传递，即一个派生类不允许它的直接或间接派生类继承或定义这个虚函数。此时可以在当前类的虚函数签名后添加final关键字。表示虚函数的继承到此为止，其派生类不能再定义或继承该虚函数了。

```
class Y : public X {  
public:  
    virtual void print() override final { } // 阻止派生类重写 print()  
};
```

final 类

- 定义一个类时，可以在类名后用关键字final将一个类定义为final类（最终类），即不能再从这个类定义任何派生类。

```
class Y final : public X {  
public:  
    virtual void print() override { }  
};
```

表示不能从Y类定义派生类，或者说任何类不能将Y作为基类。

10.10 多继承和虚基类

- C++支持多继承（Multiple Inheritance），即一个派生类可以同时继承多个直接基类。
- 多继承允许派生类整合多个基类的特性，从而实现更复杂、功能更丰富的类层次结构。
- 例如，一个TeachingAssistant（助教）类可能需要同时具备Student（学生）和Teacher（教师）的特性。

```
class D : public A, protected B, private C {  
    // D类的成员  
};
```

```
class Shape {
public:
    virtual void draw() { std::cout << "绘制一般形状\n"; }
};

class Color {
    std::string color_name;
public:
    Color(const std::string& name) : color_name(name) {}
    std::string get_color() const { return color_name; }
};

class Circle : public Shape, public Color {
public:
    Circle(const std::string& color_name) : Color(color_name) {} // 构造函数必须初始化 Color
    void draw() override { std::cout << "绘制圆\n"; }
};

int main() {
    Circle c("红色");
    std::cout << "颜色: " << c.get_color() << '\n';
    c.draw();
    c.Shape::draw(); //调用Shape的draw()函数

    return 0;
}
```

二义性问题

- 多继承可能导致二义性问题，即当多个基类包含同名成员（数据或函数）时，派生类对象访问这些成员会引发歧义。

```
class USBDevice {
private:
    long m_id;
public:
    USBDevice(long id) : m_id(id) {}
    long getID() const { return m_id; }
};

class NetworkDevice {
private:
    long m_id;
public:
    NetworkDevice(long id) : m_id(id) {}
    long getID() const { return m_id; }
};

class WirelessAdapter : public USBDevice, public NetworkDevice {
public:
    WirelessAdapter(long usbId, long networkId)
        : USBDevice(usbId), NetworkDevice(networkId) {}
};
```

```

class USBDevice {
private:
    long m_id;
public:
    USBDevice(long id) : m_id(id) {}
    long getID() const { return m_id; }
};

class NetworkDevice {
private:
    long m_id;
public:
    NetworkDevice(long id) : m_id(id) {}
    long getID() const { return m_id; }
};

class WirelessAdapter : public USBDevice, public NetworkDevice {
public:
    WirelessAdapter(long usbId, long networkId)
        : USBDevice(usbId), NetworkDevice(networkId) {}
};

```

通过作用域解析运算符 (::)
显式指定基类可以解决歧义性
问题

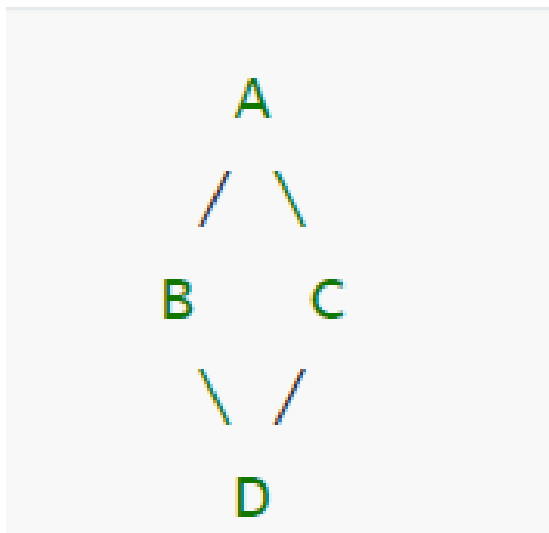
```

int main() {
    WirelessAdapter wa(5442, 181742);
    // std::cout << wa.getID(); // 错误: 调用不明确
    std::cout << "USB ID: " << wa.USBDevice::getID() << std::endl;
    std::cout << "Network ID: " << wa.NetworkDevice::getID() << std::endl;
}

```

10.10.2 虚继承与菱形继承

- 菱形继承（Diamond Inheritance）是多继承中常见的问题，当一个派生类通过多条路径继承同一基类时，可能导致基类成员重复和二义性。



```
class A {  
public:  
    int a;  
    A() : a(1) { std::cout << "A的构造函数\n"; }  
    void showA() { std::cout << "A类中的成员a: " << a << '\n'; }  
};
```

```
class B : public A {  
public:  
    B() { std::cout << "B的构造函数\n"; }  
};
```

```
class C : public A {  
public:  
    C() { std::cout << "C的构造函数\n"; }  
};
```

```
class D : public B, public C {  
public:  
    D() { std::cout << "D的构造函数\n"; }  
};
```

```
int main() {  
    D d;  
    d.showA(); // 错误: 无法调用showA, 因为有二义性  
    return 0;  
}
```

A的构造函数

B的构造函数

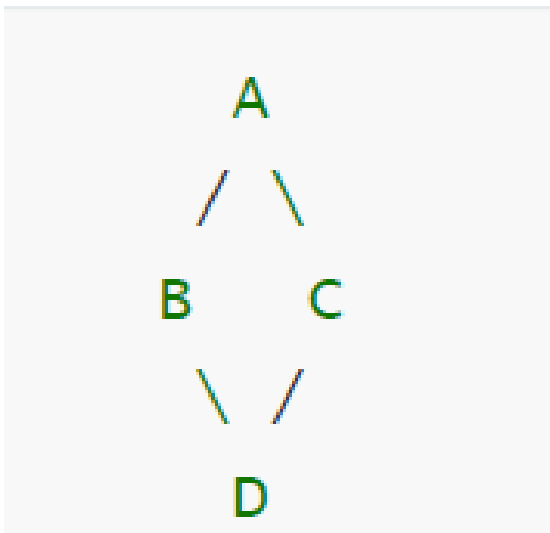
A的构造函数

C的构造函数

D的构造函数

使用虚继承解决菱形继承问题

- 为了解决菱形继承带来的重复继承问题，C++提供了虚继承（Virtual Inheritance）。
- 虚继承的目的是确保通过多个路径继承自同一个基类时，派生类只有一个基类的实例，从而避免了重复继承和二义性。



虚继承的基本语法

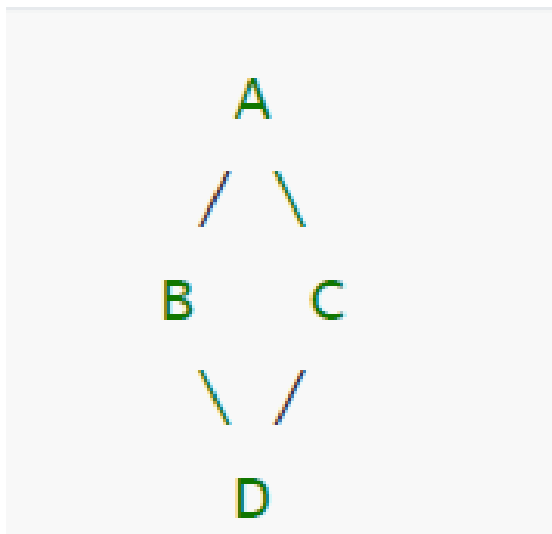
在使用虚继承时，需要在继承声明中使用 `virtual` 关键字。例如，如果 `B` 和 `C` 都要虚继承 `A`，则可以这样声明：

```
class B : virtual public A { ... };  
class C : virtual public A { ... };
```

由于 `B` 和 `C` 都是虚继承于 `A`，从而 `D` 只包含一个 `A` 的实例，`B` 和 `C` 共享同一份 `A`，避免了重复继承的问题。

虚继承的构造函数调用顺序

- 在虚继承中，最底层的派生类（这里是D）负责调用虚基类（A）的构造函数，而非中间基类（B和C）。这保证了虚基类只初始化一次。



```
class A {  
public:  
    int a;  
    A() : a(1) { std::cout << "A的构造函数\n"; }  
};
```

```
class B : virtual public A {  
public:  
    B() { std::cout << "B的构造函数\n"; }  
};
```

```
class C : virtual public A {  
public:  
    C() { std::cout << "C的构造函数\n"; }  
};
```

```
class D : public B, public C {  
public:  
    D() { std::cout << "D的构造函数\n"; }  
};
```

```
int main() {  
    D d; // 创建D类的对象，观察构造函数调用顺序  
    return 0;  
}
```

程序执行结果:

A的构造函数
B的构造函数
C的构造函数
D的构造函数

10.11 纯虚函数与抽象类

- 类不仅用于描述对象的属性和行为，还可定义接口，规定派生类的行为。**纯虚函数**和**抽象类**是实现这一目标的关键工具，广泛应用于面向对象编程（OOP）中，特别是在需要多态和接口设计的场景。

10.11.1 纯虚函数

- 纯虚函数是没有实现的虚函数，仅定义接口，强制派生类提供具体实现。其声明使用= 0标识。通过声明纯虚函数，基类要求派生类必须提供该函数的实现。

```
class Base {  
public:  
    virtual void someFunction() = 0;    // 纯虚函数  
};
```

10.11.2 抽象类

- 抽象类是包含至少一个纯虚函数的类，不能直接实例化，主要用作基类，提供通用接口。

```
class Base {  
public:  
    virtual void someFunction() = 0; // 纯虚函数  
};
```

```
Base b; // 错误：无法实例化抽象类
```

10.11.3 派生类的实现

- 派生类继承抽象类时，必须实现所有纯虚函数，否则仍为抽象类，无法实例化。只有实现了所有纯虚函数的类才能被实例化。

```
class Shape {  
public:  
    virtual void draw() = 0; // 纯虚函数  
    virtual double area() = 0; // 纯虚函数  
};
```

```
class Shape {
public:
    virtual void draw() = 0; // 纯虚函数
    virtual double area() = 0; // 纯虚函数
};
```

```
class Circle : public Shape {
public:
    void draw() override {
        std::cout << "绘制圆形\n";
    }

    double area() override {
        return 3.14 * radius * radius;
    }

    Circle(double r) : radius(r) {}

private:
    double radius;
};
```

如果派生类没有实现所有纯虚函数，编译器将报错，提示该类仍然是一个抽象类，无法实例化。

10.11.4 抽象类的作用

- 1. 定义接口： 抽象类用于定义一组接口，确保所有派生类遵循相同的行为规范。例如，可以为所有图形对象（如圆形、矩形、三角形等）定义一个共同的基类Shape，并强制所有派生类实现draw()和area()等方法。

```
class Shape {  
public:  
    virtual void draw() = 0; // 纯虚函数  
    virtual double area() = 0; // 纯虚函数  
};
```

10.11.4 抽象类的作用

- 2. 通用编程：通过抽象类，可以编写不依赖于具体类型的代码，只依赖于接口。例如，可以编写一个函数，该函数接受一个 `Shape*` 类型的参数，意味着该函数可以接受任何继承自 `Shape` 的对象，而无需知道具体的派生类。

```
void printArea(Shape* shape) {  
    std::cout << "面积: " << shape->area() << std::endl;  
}
```

10.11.4 抽象类的作用

- 3. 多态：使用抽象类时，常常涉及到多态性。通过基类指针或引用，可以操作不同派生类的对象，发挥出多态的优势。例如，以下代码展示了如何使用Shape*指针来调用不同派生类的draw()和area()方法：

```
Shape* shapes[] = {new Circle(5), new Rectangle(4, 6)};
for (Shape* shape : shapes) {
    shape->draw();    // 调用不同的实现
    std::cout << "面积： " << shape->area() << std::endl;
}
```

纯虚函数可以有函数体吗？

```
#include <print>

class Animal {
public:
    virtual void speak() = 0;
};

// 类外定义纯虚函数
void Animal::speak() {
    std::println("动物发出声音");
}

class Dog : public Animal {
public:
    void speak() override {
        Animal::speak(); // 调用基类的实现
        std::println("汪汪! ");
    }
};
```

```
int main() {
    Dog d;
    d.speak(); // 输出：动物发出声音 \n 汪汪!
    return 0;
}
```

纯虚析构函数必须提供定义

```
class Base {
public:
    virtual ~Base() = 0;    // 纯虚析构函数
};

Base::~~Base() {           // 必须在类外定义
    std::println("Base析构");
}

class Derived : public Base {
public:
    ~Derived() {
        std::println("Derived析构");
    }
};
```

```
int main() {
    Derived d;
    // 输出顺序: Derived析构 → Base析构
    return 0;
}
```

纯虚析构函数是一个特例，因为它必须有一个实现

总结

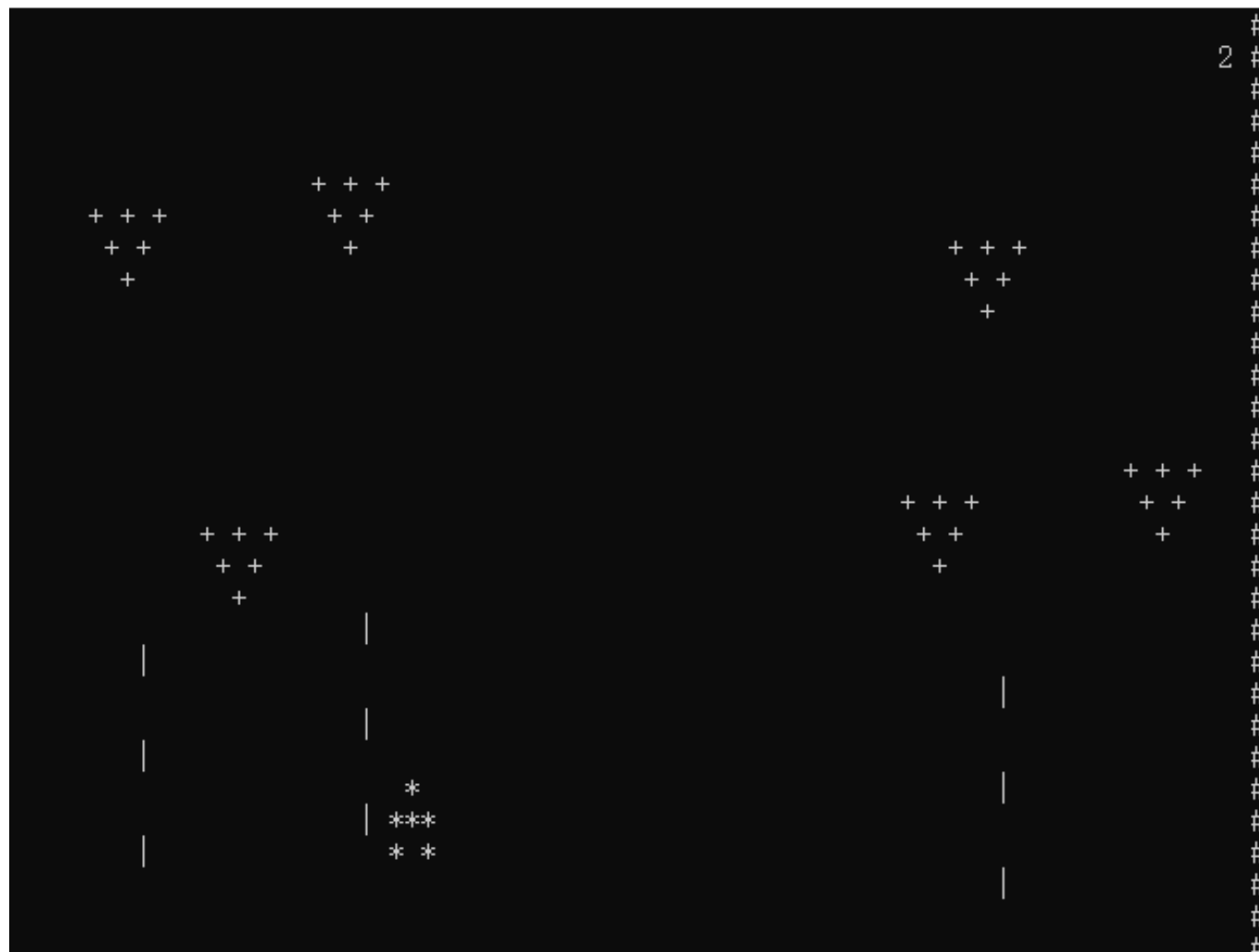
C++的继承机制以派生类为核心，支持代码复用和扩展：

- **继承方式**：通过public、protected和private控制基类成员访问。
- **构造函数**：派生类构造函数需调用基类构造函数，确保正确初始化。
- **虚函数**：通过virtual实现运行时多态，通过基类指针或引用可调用派生类方法。
- **虚析构函数**：基类析构函数应为virtual，确保派生类资源正确释放。
- **多继承**：支持从多个基类继承，菱形继承问题通过**虚继承**解决。
- **纯虚函数与抽象类**：以= 0定义接口，派生类需实现所有纯虚函数才能实例化。

面向对象的仿“雷电战机”游戏

面向对象的仿“雷电战机”游戏

F:\hwdong_courses\Cplusplus\spaceInvader\Debug\spaceInvader.exe



面向对象的仿“雷电战机”游戏

<https://github.com/hwdong-net/cplusplus17/blob/master/projects/spaceInvader.cpp>

关注

Youtube频道: <https://www.youtube.com/c/hwdong>

博客: <https://hwdong-net.github.io>

B站: hw-dong